Unit II

2. Programming with PHP

2.1 Conditional statements: if, if-else, switch, The ? Operator
2.2 Looping statements: while Loop, do...while Loop, for Loop
2.3 Arrays in PHP: Introduction- What is Array?
2.4 Types of Arrays: Indexed Vs. Associative arrays, Multidimensional arrays
2.5 Creating Array, Accessing Array, Manipulating Arrays, Displaying arrays…..use of for.. each as loop
2.6 Using Array Functions
2.7 Including and Requiring Files- use of Include() and Require()
2.8 Implicit and Explicit Casting in PHP

2.1 Conditional statements: if, if-else, switch, The ? Operator:

Conditional Statements in PHP:

Conditional statements allow PHP to make **decisions** and execute certain blocks of code **based on conditions**.

1. `if` Statement:

Executes a block of code **if the condition is true**.

php

```
<?php
    $age = 18;
    if ($age >= 18) {
        echo "You can vote.";
    }
?>
```

2. `if-else` Statement:

Provides an **alternative block** if the condition is false.

php

```
<?php
    if ($age >= 18) {
        echo "Adult";
    } else {
        echo "Minor";
    }
?>
```

3. `switch` Statement:

Used to **select one block of code from many** options.

php

```
<?php
    $day = "Monday";
    switch ($day) {
        case "Monday":
            echo "Start of the week";
```

```
                break;
            case "Friday":
                echo "Weekend is near";
                break;
            default:
                echo "Normal day";
        }
    ?>
```

4. Ternary (`?`) Operator:

A **short-hand** way to write an `if-else` condition.

Syntax:

php

```
(condition) ? value_if_true : value_if_false;
```

Example:

php

```
<?php
    $marks = 75;
    $result = ($marks >= 40) ? "Pass" : "Fail";
    echo $result;
?>
```

2.2 Looping statements: while Loop, do...while Loop, for Loop:

Looping Statements in PHP:

Loops are used to **execute a block of code repeatedly** as long as a specific condition is true.

1. `while` Loop:

Repeats code **as long as the condition is true**.

php

```
<?php
    $i = 1;
    while ($i <= 5) {
        echo $i . " ";
        $i++;
    }
?>
```

Checks condition before executing the loop.

2. `do...while` Loop:

Executes the code **at least once**, then checks the condition.

php

```
<?php
    $i = 1;
```

```
do {
    echo $i . " ";
    $i++;
} while ($i <= 5);
?>
```

Always runs the loop once, even if condition is false.

3. `for` Loop:

Best when the number of repetitions is **known**.

php

```
<?php
    for ($i = 1; $i <= 5; $i++) {
        echo $i . " ";
    }
?>
```

All loop steps (initialization, condition, increment) are in one line.

Summary Table:

Loop	Condition Check	Best Use
`while`	Before loop	Unknown repetitions
`do…while`	After first run	At least one execution needed
`for`	Before loop	Known number of repetitions

2.3 Arrays in PHP: Introduction- What is Array?

What is an Array?

An **array** in PHP is a special variable that can hold **multiple values** under a **single name**. Arrays are useful when you want to store and manage a list of items (like names, numbers, etc.).

1. Why Use Arrays?

To store a collection of data.
Makes code more organized and easier to manage.
Allows looping through values.

2. Types of Arrays in PHP:

a. Indexed Arrays (Numeric Index):

Uses numbers (starting from 0) as keys.

php

```
<?php
    $fruits = array("Apple", "Banana", "Mango");
    echo $fruits[1]; // Outputs: Banana
?>
```

b. Associative Arrays:

Uses **named keys** instead of numbers.

php

```
<?php
    $person = array("name" => "Ali", "age" => 25);
    echo $person["name"]; // Outputs: Ali
?>
```

c. Multidimensional Arrays:

Arrays within arrays (like a table or matrix).

php

```
<?php
    $marks = array(
        array(85, 90),
        array(78, 88)
    );
    echo $marks[0][1]; // Outputs: 90
?>
```

3. Advantages of Arrays:

Store large amounts of data.
Easy to sort, loop, and search.
Supports both numbers and string keys.

2.4 Types of Arrays: Indexed Vs. Associative arrays, Multidimensional arrays:

Types of Arrays in PHP:

PHP supports **three main types of arrays**:

1. Indexed Arrays:

Uses **numeric indexes** (starting from 0).
Stores values in a linear order.

Example:

php

```
<?php
    $colors = array("Red", "Green", "Blue");
    echo $colors[1]; // Output: Green
?>
```

Best when values are ordered and don't need labels.

2. Associative Arrays:

Uses **named keys** instead of numbers.
Stores data in **key-value pairs**.

Example:

php

```php
<?php
    $student = array("name" => "Amina", "age" => 20);
    echo $student["name"]; // Output: Amina
?>
```

Useful when data has labels (like name, age, price).

3. Multidimensional Arrays:

Arrays **inside other arrays**.
Used to represent **tables or complex data**.

Example:

php

```php
<?php
    $marks = array(
        "Ali" => array("Math" => 90, "Science" => 85),
        "Sara" => array("Math" => 95, "Science" => 80)
    );
    echo $marks["Ali"]["Science"]; // Output: 85
?>
```

Helpful for tabular data like student marks, products, etc.

Summary Table:

Type	Index Type	Example Index	Use Case
Indexed Array	Numeric (0, 1, 2…)	`$array[0]`	List of items
Associative Array	Named keys	`$array["name"]`	Key-value data
Multidimensional	Array of arrays	`$array[0][1]`	Table-like data

2.5 Creating Array, Accessing Array, Manipulating Arrays, Displaying arrays…..use of for..each as loop:

1. Creating Arrays in PHP:

Arrays can be created using the `array()` function or `[]` (short syntax).

Examples:

php

```php
$colors = array("Red", "Green", "Blue");
$fruits = ["Apple", "Banana", "Mango"];
```

2. Accessing Array Elements:

Elements are accessed using their **index** or **key**.

php

```php
echo $colors[1]; // Output: Green
```

php

```
$person = ["name" => "Ali", "age" => 25];
echo $person["name"]; // Output: Ali
```

3. Manipulating Arrays:

You can **add**, **modify**, or **remove** array elements.

Add:

php

```
$colors[] = "Yellow"; // Adds Yellow to the array
```

Modify:

php

```
$colors[0] = "Pink"; // Changes Red to Pink
```

Remove:

php

```
unset($colors[1]); // Removes Green
```

4. Displaying Arrays:

Use `print_r()` or `var_dump()` for quick display:

php

```
print_r($colors);
```

For custom display, use a loop.

5. Use of `foreach` Loop:

`foreach` is used to iterate over **arrays easily**.

Indexed Array:

php

```
foreach ($colors as $color) {
    echo $color . "<br>";
}
```

Associative Array:

php

```
foreach ($person as $key => $value) {
    echo "$key: $value <br>";
}
```

Summary:

Arrays store multiple values.
`foreach` is the best way to loop through arrays.
Arrays can be created, accessed, changed, and displayed easily in PHP.

2.6 Using Array Functions:

Using Array Functions in PHP:

PHP provides many **built-in array functions** to work with arrays easily — such as **adding, removing, sorting, counting**, etc.

Commonly Used Array Functions:

`count()`:

Returns the **number of elements** in an array.

php

```
$colors = ["Red", "Green", "Blue"];
echo count($colors); // Output: 3
```

`array_push()`:

Adds elements to the **end** of an array.

php

```
array_push($colors, "Yellow");
```

`array_pop()`:

Removes the **last** element from an array.

php

```
array_pop($colors);
```

`array_merge()`:

Combines two or more arrays into one.

php

```
$fruits = ["Apple", "Banana"];
$veggies = ["Carrot", "Tomato"];
$items = array_merge($fruits, $veggies);
```

`sort()`:

Sorts an array in **ascending order**.

php

```
sort($colors);
```

Why Use Array Functions?

Save time and effort
Clean and optimized code
Make array handling easier

Example:

php

```
$numbers = [4, 2, 8, 1];
sort($numbers);
foreach ($numbers as $num) {
    echo $num . " ";
}
```

// Output: 1 2 4 8

2.7 Including and Requiring Files- use of Include() and Require():

Including and Requiring Files in PHP:

PHP allows you to **reuse code** by including other PHP files into your current script using **`include()`** and **`require()`**.

1. Why Use `include()` or `require()`?

Avoid code repetition.
Keep your code organized (e.g., separating header, footer, config files).
Easy to manage large projects.

2. `include()` Function:

Includes and evaluates a file.
Continues execution even if the file is not found (shows a warning).

Example:

```
php
    include("header.php");
    echo "Main Page Content";
```

3. `require()` Function:

Includes and evaluates a file.
Stops execution if the file is not found (shows a fatal error).

Example:

```
php
    require("config.php");
```

Key Differences:

Function	If file not found	Use Case
`include()`	Shows warning, continues	Non-critical files (e.g., sidebar)
`require()`	Fatal error, stops	Critical files (e.g., DB config)

4. Best Practices:

Use `require()` for important files (like database connections).
Use `include()` for optional parts (like footer, sidebar).

2.8 Implicit and Explicit Casting in PHP:

Type Casting in PHP:

Type casting means converting a variable from one data type to another. PHP supports both:

Implicit Casting (automatic)
Explicit Casting (manual)

1. Implicit Casting (Type Juggling):

PHP **automatically converts** data types when needed.
Happens during operations like arithmetic or string concatenation.

Example:

php

```
$x = "5" + 10; // "5" is automatically cast to integer
echo $x; // Output: 15
```

Useful when combining strings and numbers.

2. Explicit Casting (Manual Conversion):

You **manually convert** a value using casting syntax.
Format: `(data_type) $variable`

Example:

php

```
$val = "123.45";
$intVal = (int)$val;
echo $intVal; // Output: 123
```

Ensures the variable is treated as a specific type.

3. Common Casting Types:

Type	Syntax
Integer	`(int)$var`
Float	`(float)$var`
String	`(string)$var`
Boolean	`(bool)$var`
Array	`(array)$var`

Why It Matters:

Prevents bugs when working with mixed types.
Helps control how data behaves in operations.